Intelligent Robots

LAB3

郝祁 Hao Qi
Email:haoq@sustech.edu.cn

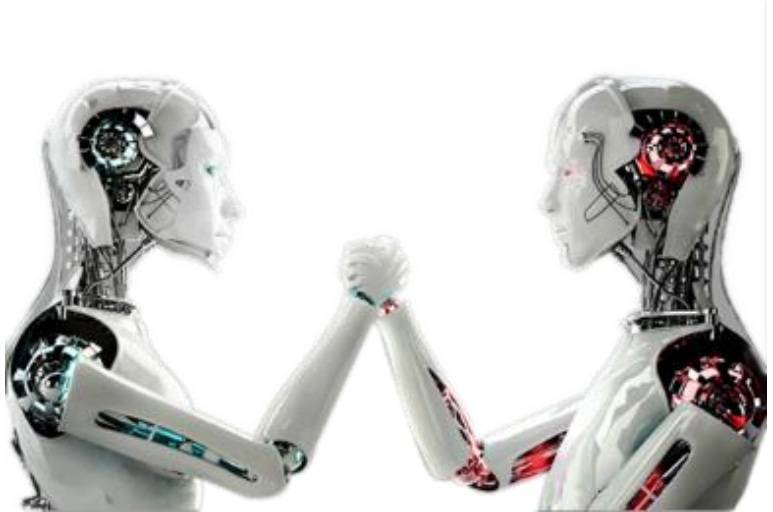
OBJECTIVES

01 TF

02 Visualization tools

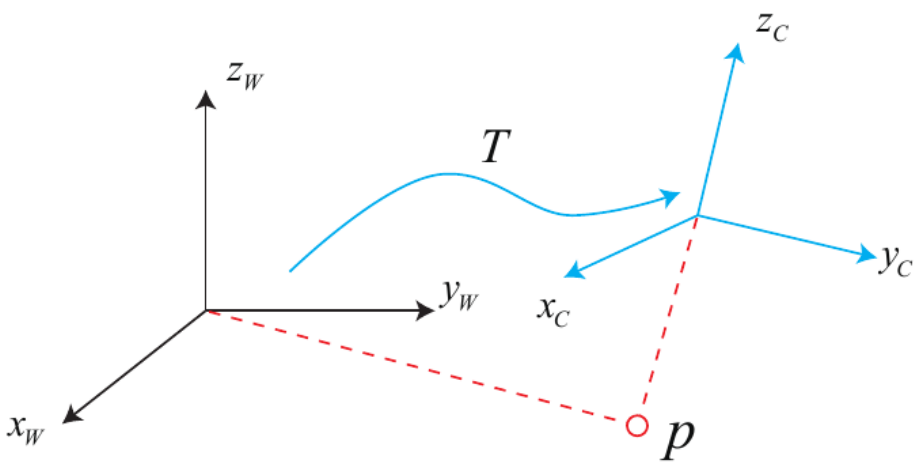
03 Lab Task

PART ONE



tf

➢ Coordinate transformation



$$\boldsymbol{a}' = \boldsymbol{R}\boldsymbol{a} + \boldsymbol{t}.$$

The motion between the two frames can be completely described by R,t.

R rotation matrix, and t translation vector

- Homogeneous(齐次形式)

$$\begin{bmatrix} \boldsymbol{a}' \\ 1 \end{bmatrix} = \begin{bmatrix} \boldsymbol{R} & \boldsymbol{t} \\ \boldsymbol{0}^T & 1 \end{bmatrix} \begin{bmatrix} \boldsymbol{a} \\ 1 \end{bmatrix} \triangleq \boldsymbol{T} \begin{bmatrix} \boldsymbol{a} \\ 1 \end{bmatrix}.$$

$$\tilde{\boldsymbol{b}} = \boldsymbol{T}_1 \tilde{\boldsymbol{a}},\ \tilde{\boldsymbol{c}} = \boldsymbol{T}_2 \tilde{\boldsymbol{b}} \quad \Rightarrow \tilde{\boldsymbol{c}} = \boldsymbol{T}_2 \boldsymbol{T}_1 \tilde{\boldsymbol{a}}.$$

tf

➢ Coordinate system in robot

A robotic system typically has many **3D coordinate frames** that change over time, such as a **world frame**, **base frame**, **gripper frame**, **head frame**, etc.

Look at the PR2 robot. If he want to take a bottle of beer, what programs inside should run?

- Sensors locates the beer
- Arm manipulated to the beer
- Grab the bottle
- Take it out
- …

Arm should know the coordinates of beer relative to itself, not just the sensors.

tf keeps track of all these frames over time.

tf

What is tf?

tf is a package that lets the user keep track of multiple coordinate frames over time.

What does tf do?

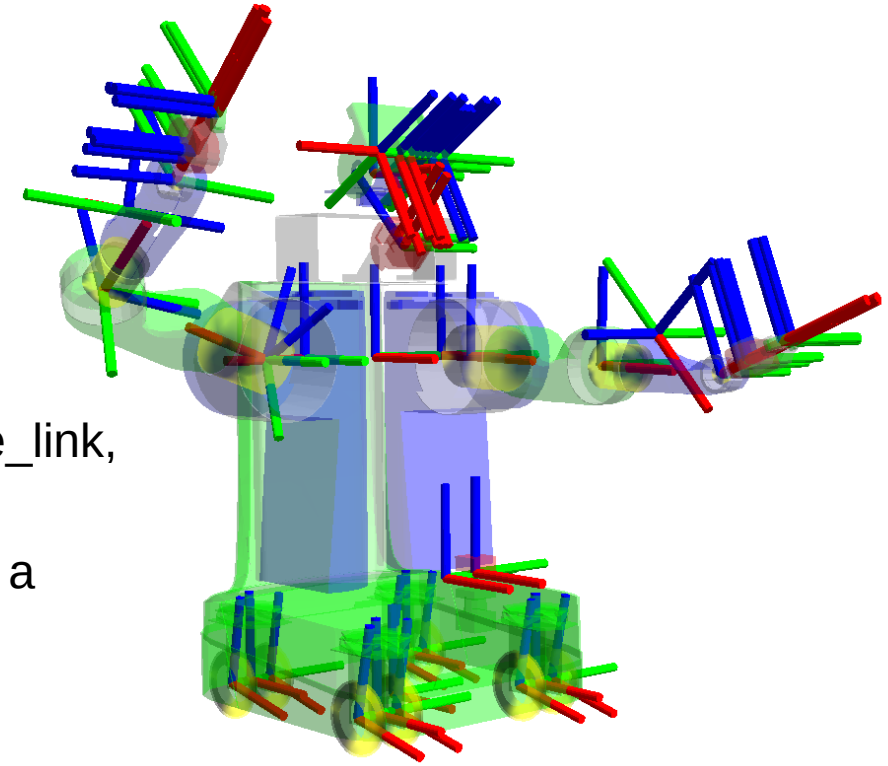
- tf maintains the relationship between coordinate frames in a tree structure buffered in time,
- tf lets the user transform points, vectors, etc between any two coordinate frames at any desired point in time.

How does TF work?

- Listening for transforms
- Broadcasting transforms

Special terms

- **Link** – each component is a link, for example: base_link, left_wheel_link, camera_link etc.
- **Frame** – each link corresponds a frame, a frame is a coordinate
- **Joint** – the connection between two links.

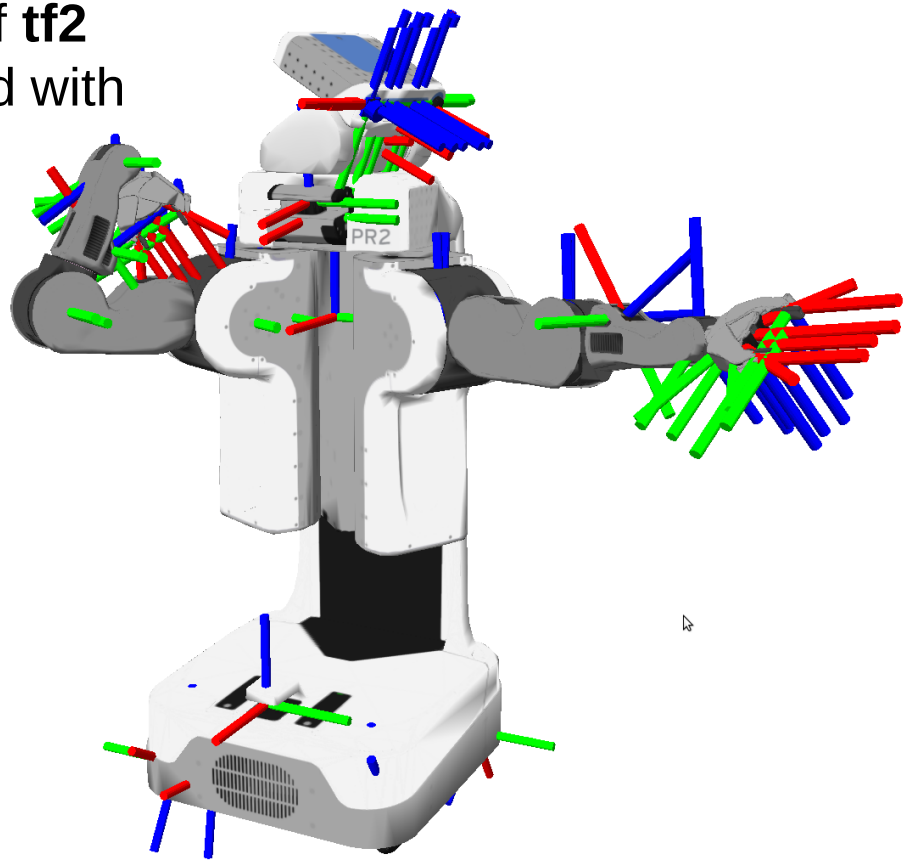


http://wiki.ros.org/tf

tf

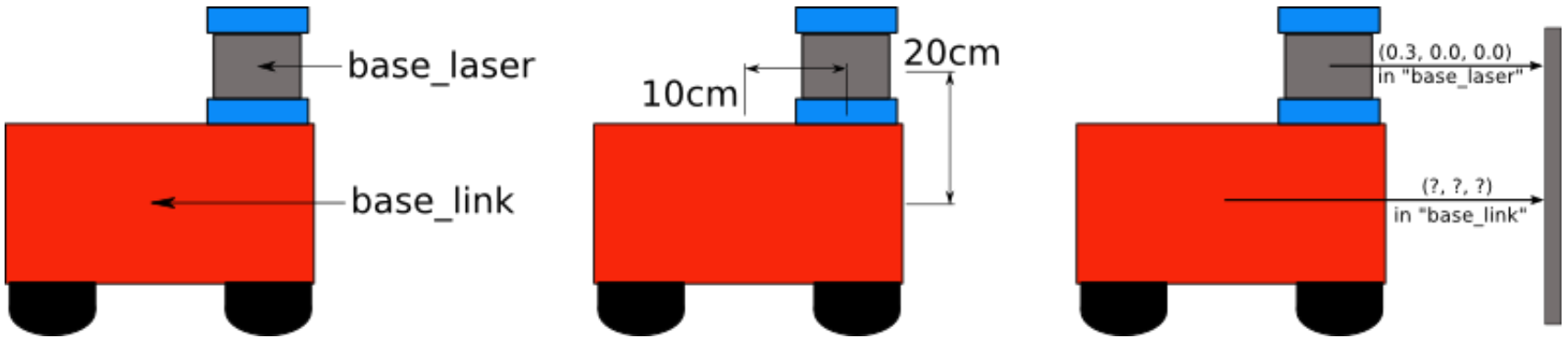
➢ tf2

- **tf2** is the second generation of the transform library.
- Since ROS Hydro, **tf** has been "deprecated" in favor of **tf2**
- The **tf** implementation has been removed and replaced with calls to **tf2**

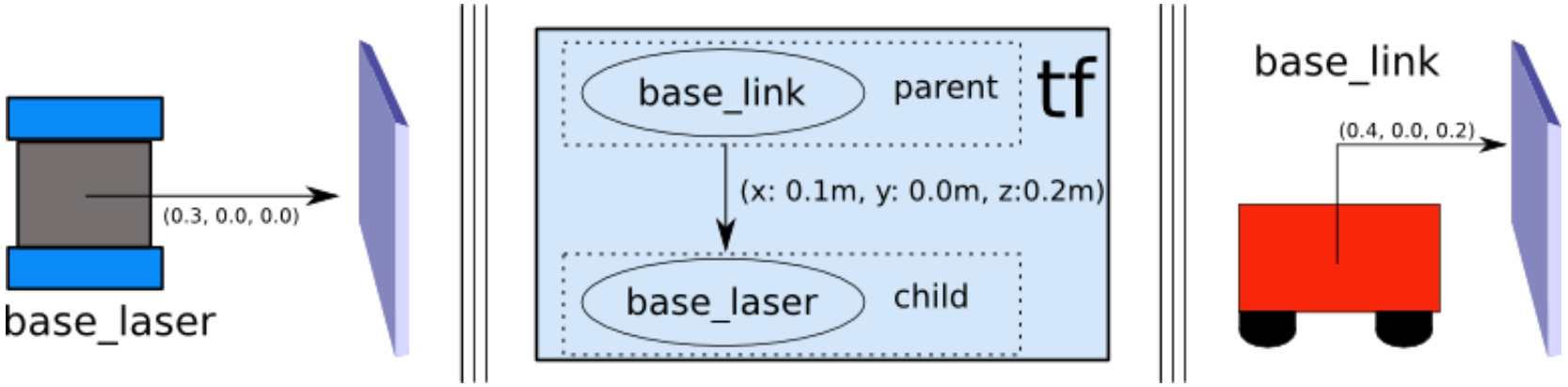


http://wiki.ros.org/tf2

➢ Coordinate transformation of a simple mobile robot



The coordinate frame of the base_link and the base_laser



The relationship between the "base_laser" and "base_link" coordinate frames.

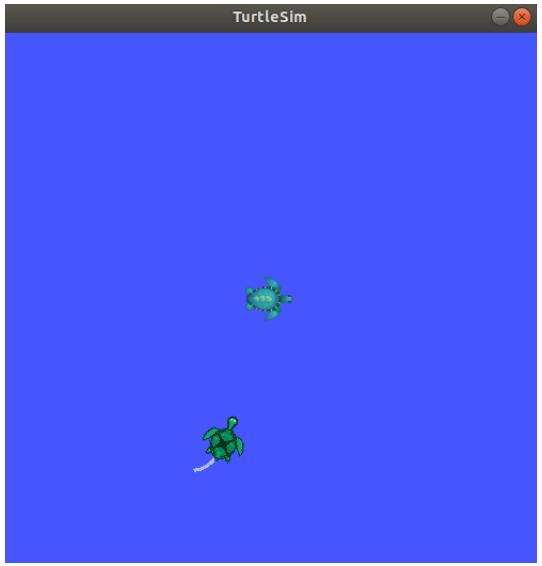
tf

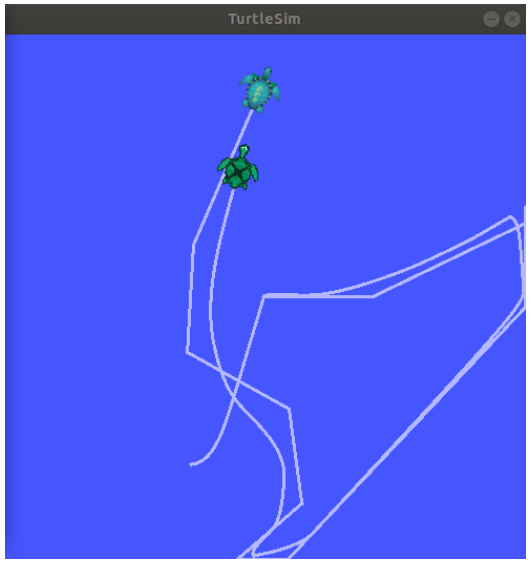
➢ tf introduction demo

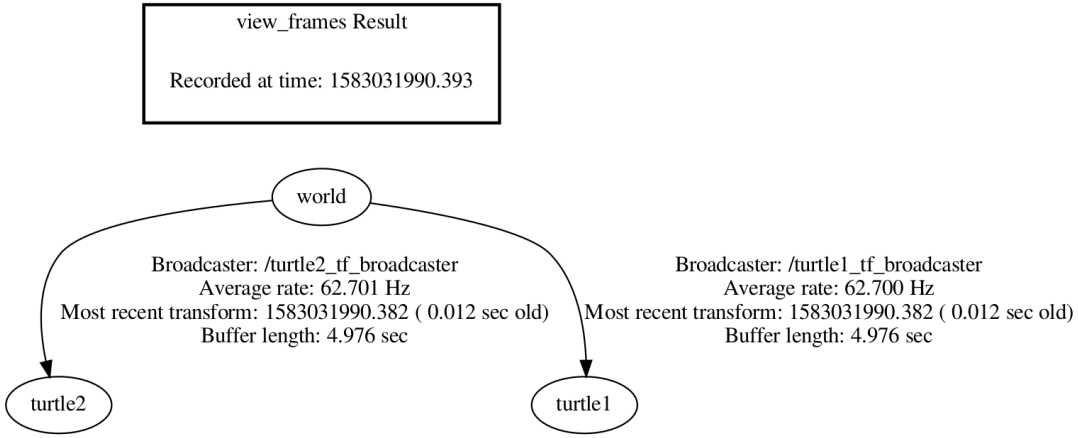
```
$sudo apt-get install ros-melodic-ros-tutorials ros-melodic-geometry-tutorials ros-melodic-rviz ros-melodic-rosbash ros-melodic-rqt-tf-tree
$roslaunch turtle_tf turtle_tf_demo.launch
$rosrun turtlesim turtle_teleop_key
$rosrun tf view_frames or rosrun rqt_tf_tree rqt_tf_tree
```

view_frames creates a diagram of the frames being broadcast by tf over ROS.

rqt_tf_tree is a runtime tool for visualizing the tree of frames being broadcast over ROS.







tf

➢ tf introduction demo 1

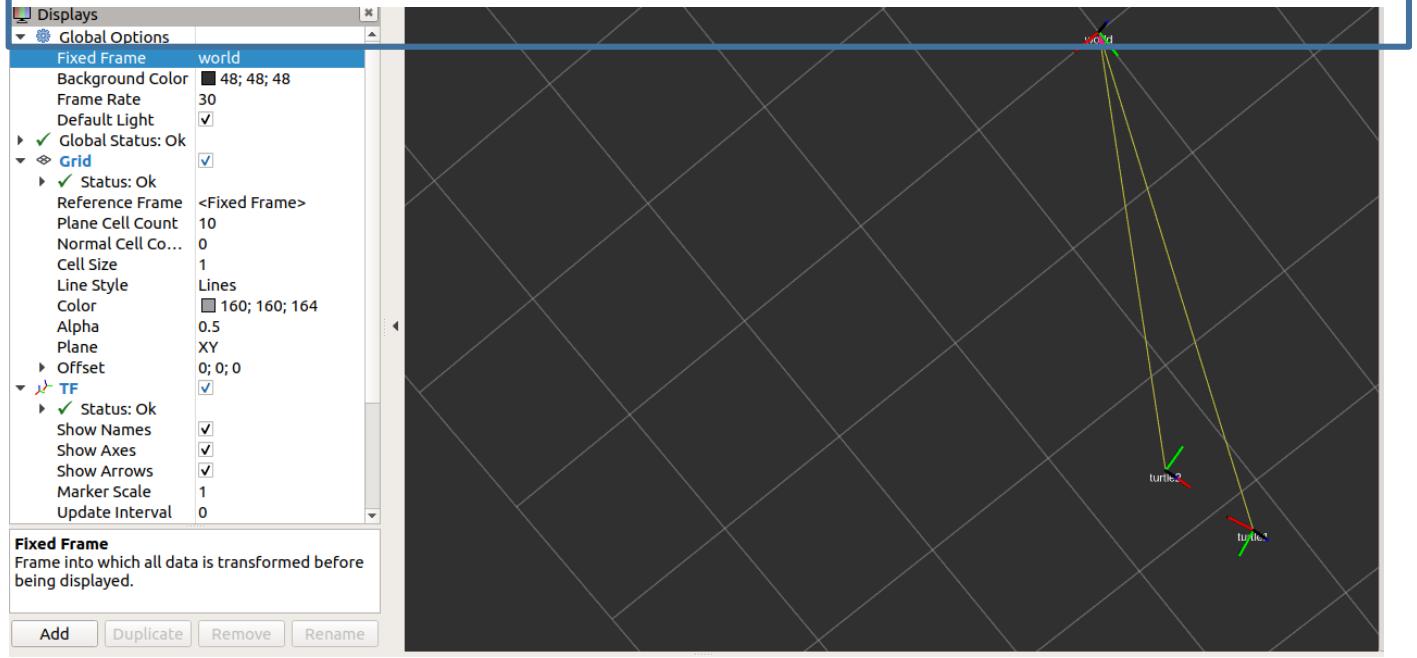
```
$rosrun tf tf_echo turtle1 turtle2
```

tf_echo reports the transform between any two frames broadcast over ROS.

$$\mathbf{T}_{turtle1_turtle2} = \mathbf{T}_{turtle1_world} * \mathbf{T}_{world_turtle2}$$

```
jyh@jyh:~/catkin_ws$ rosrun tf tf_echo turtle1 turtle2
At time 1583033673.972
- Translation: [0.000, 0.000, 0.000]
- Rotation: in Quaternion [0.000, 0.000, 0.979, 0.205]
            in RPY (radian) [0.000, -0.000, 2.729]
            in RPY (degree) [0.000, -0.000, 156.361]
At time 1583033674.707
- Translation: [0.000, 0.000, 0.000]
- Rotation: in Quaternion [0.000, 0.000, 0.979, 0.205]
            in RPY (radian) [0.000, -0.000, 2.729]
            in RPY (degree) [0.000, -0.000, 156.361]
At time 1583033675.716
- Translation: [0.000, 0.000, 0.000]
- Rotation: in Quaternion [0.000, 0.000, 0.979, 0.205]
            in RPY (radian) [0.000, -0.000, 2.729]
            in RPY (degree) [0.000, -0.000, 156.361]
```

```
$ rosrun rviz rviz -d `rospack find turtle_tf`/rviz/turtle_rviz.rviz
```



rviz is a visualization tool that is useful for examining tf frames.

tf

➢ tf programming demo 2

1. Prerequisites

```
$cd ~/catkin_ws/src
$catkin_create_pkg learning_tf  tf rospy roscpp
std_msgs  turtlesim
```

2. Writing a tf broadcaster and listener

turtle_tf_broadcaster.py

```python
#!/usr/bin/env python
# -*- coding: utf-8 -*-
# This routine broadcasts tf data between world and turtle coordinates
import rospy
import tf
import turtlesim.msg

def handle_turtle_pose(msg, turtlename):
    br = tf.TransformBroadcaster()
    br.sendTransform((msg.x, msg.y, 0),
                     tf.transformations.quaternion_from_euler(0, 0, msg.theta),
                     rospy.Time.now(),
                     turtlename,
                     "world")

if __name__ == '__main__':
    rospy.init_node('turtle_tf_broadcaster')
    turtlename = rospy.get_param('~turtle')
    rospy.Subscriber('/%s/pose' % turtlename,
                     turtlesim.msg.Pose,
                     handle_turtle_pose,
                     turtlename)
    rospy.spin()
```

turtle_tf_listener.py

```python
#!/usr/bin/env python
# -*- coding: utf-8 -*-
# This routine listens for tf data ,  calculates and publishes speed instructions of turtl
import rospy
import math
import tf
import geometry_msgs.msg
import turtlesim.srv

if __name__ == '__main__':
    rospy.init_node('turtle_tf_listener')

    listener = tf.TransformListener()

    rospy.wait_for_service('spawn')
    spawner = rospy.ServiceProxy('spawn', turtlesim.srv.Spawn)
    spawner(4, 2, 0, 'turtle2')

    turtle_vel = rospy.Publisher('turtle2/cmd_vel', geometry_msgs.msg.Twist,queue_size=1)

    rate = rospy.Rate(10.0)
    while not rospy.is_shutdown():
        try:
            (trans,rot) = listener.lookupTransform('/turtle2', '/turtle1', rospy.Time(0))
        except (tf.LookupException, tf.ConnectivityException, tf.ExtrapolationException):
            continue

        angular = 4 * math.atan2(trans[1], trans[0])
        linear = 0.5 * math.sqrt(trans[0] ** 2 + trans[1] ** 2)
        cmd = geometry_msgs.msg.Twist()
        cmd.linear.x = linear
        cmd.angular.z = angular
        turtle_vel.publish(cmd)

        rate.sleep()
```

tf programming demo 2

3. Building

```
$cd ~/catkin_ws
$catkin_make
```

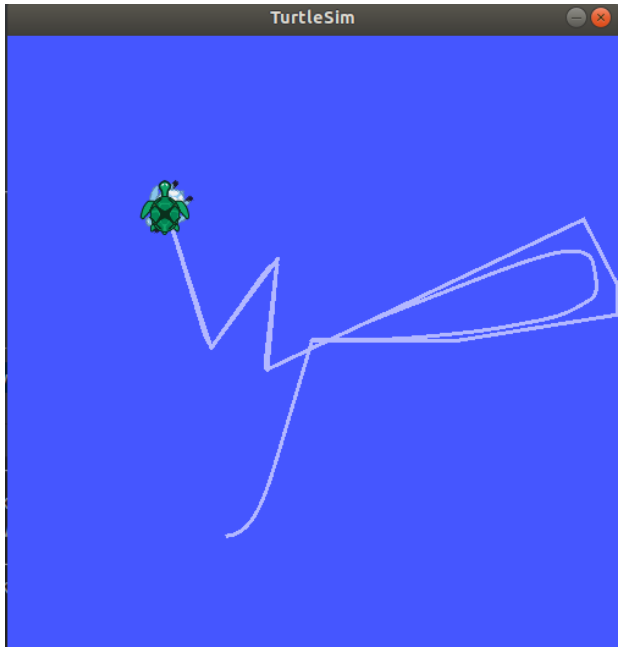
4. Create a launch file for this demo

```
<launch>

        <!-- Turtlesim Node-->
        <node pkg="turtlesim" type="turtlesim_node" name="sim"/>
        <node pkg="turtlesim" type="turtle_teleop_key" name="teleop" output="screen"/>

        <node name="turtle1_tf_broadcaster" pkg="learning_tf" type="turtle_tf_broadcaster.py" respawn="false" output="screen">
          <param name="turtle" type="string" value="turtle1" />
        </node>
        <node name="turtle2_tf_broadcaster" pkg="learning_tf" type="turtle_tf_broadcaster.py" respawn="false" output="screen">
          <param name="turtle" type="string" value="turtle2" />
        </node>

    <node pkg="learning_tf" type="turtle_tf_listener.py" name="listener" />

</launch>
```

5. Running

```
$roslaunch learning_tf learning_tf_py.launch
```

6. Checking the results

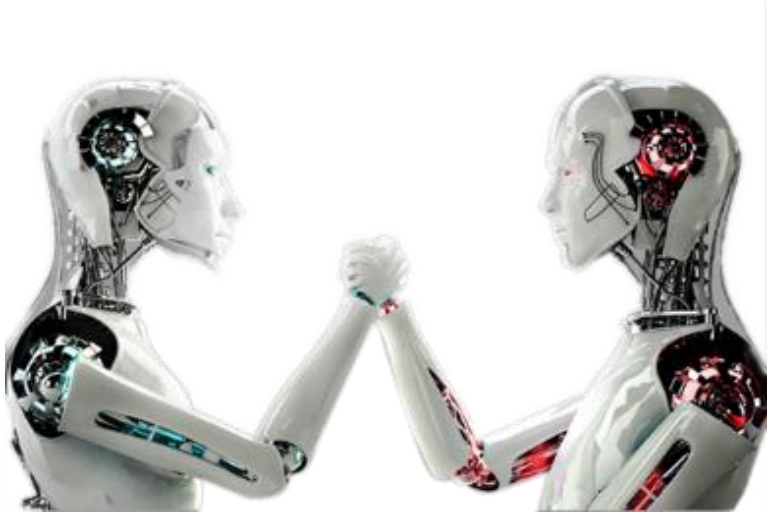


NOTE: Don't forget to make the node executable:

```
$roscd learning_tf
$chmod +x scripts/*.py
```

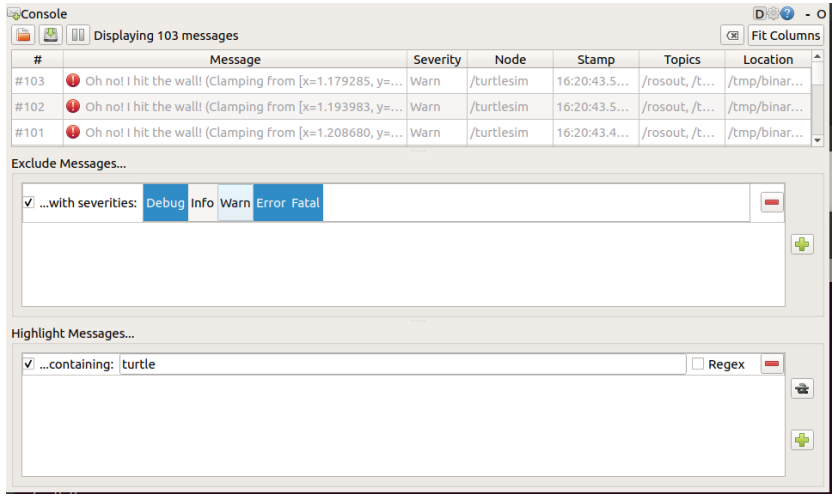
PART TWO

Visualization tools

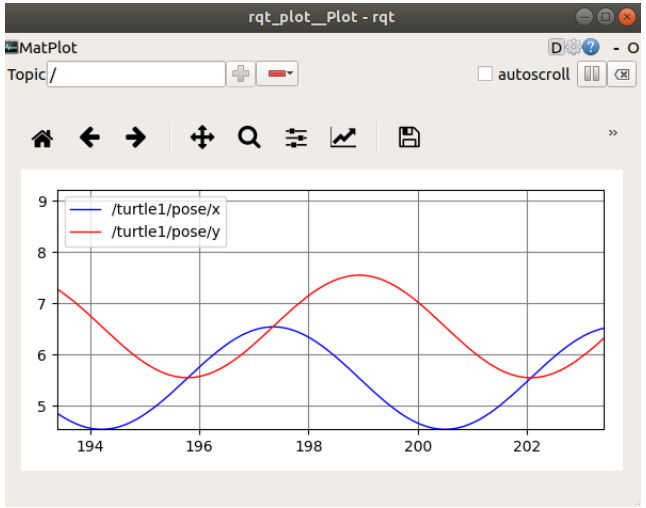


Visulization tools

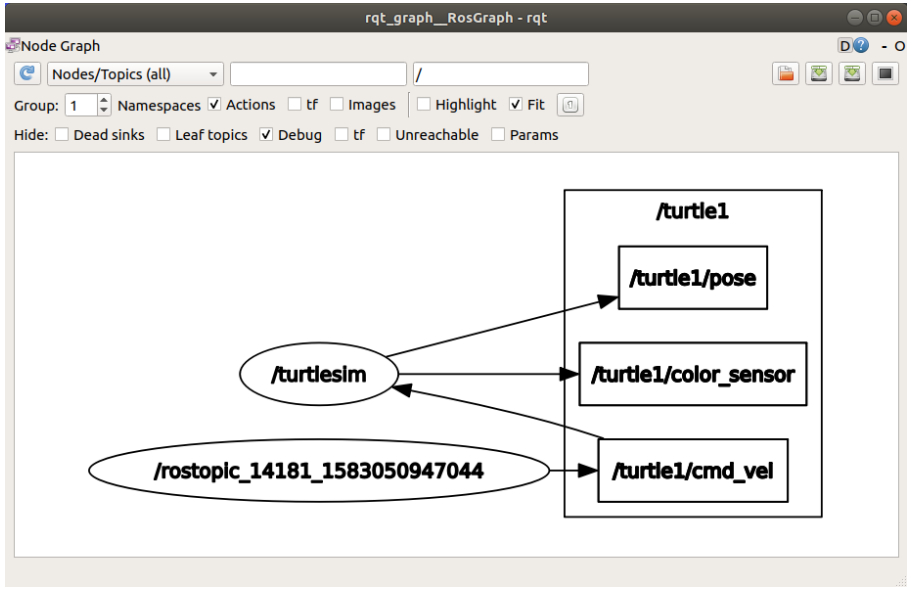
- **QT toolkit**



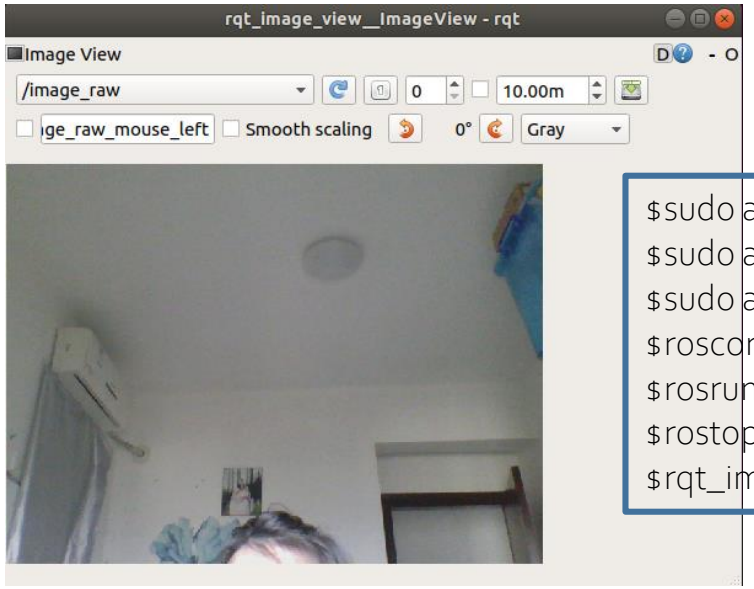
rqt_console



rqt_plot



rqt_graph



rqt_image_view

```
$sudo apt-get install ros-<ROS_DISTRO>-uvc-camera
$sudo apt-get install ros-<ROS_DISTRO>-image-*
$sudo apt-get install ros-<ROS_DISTRO>-rqt-image-view
$roscore
$rosrun uvc_camera uvc_camera_node
$rostopic list
$rqt_image_view
```

Visulization tools

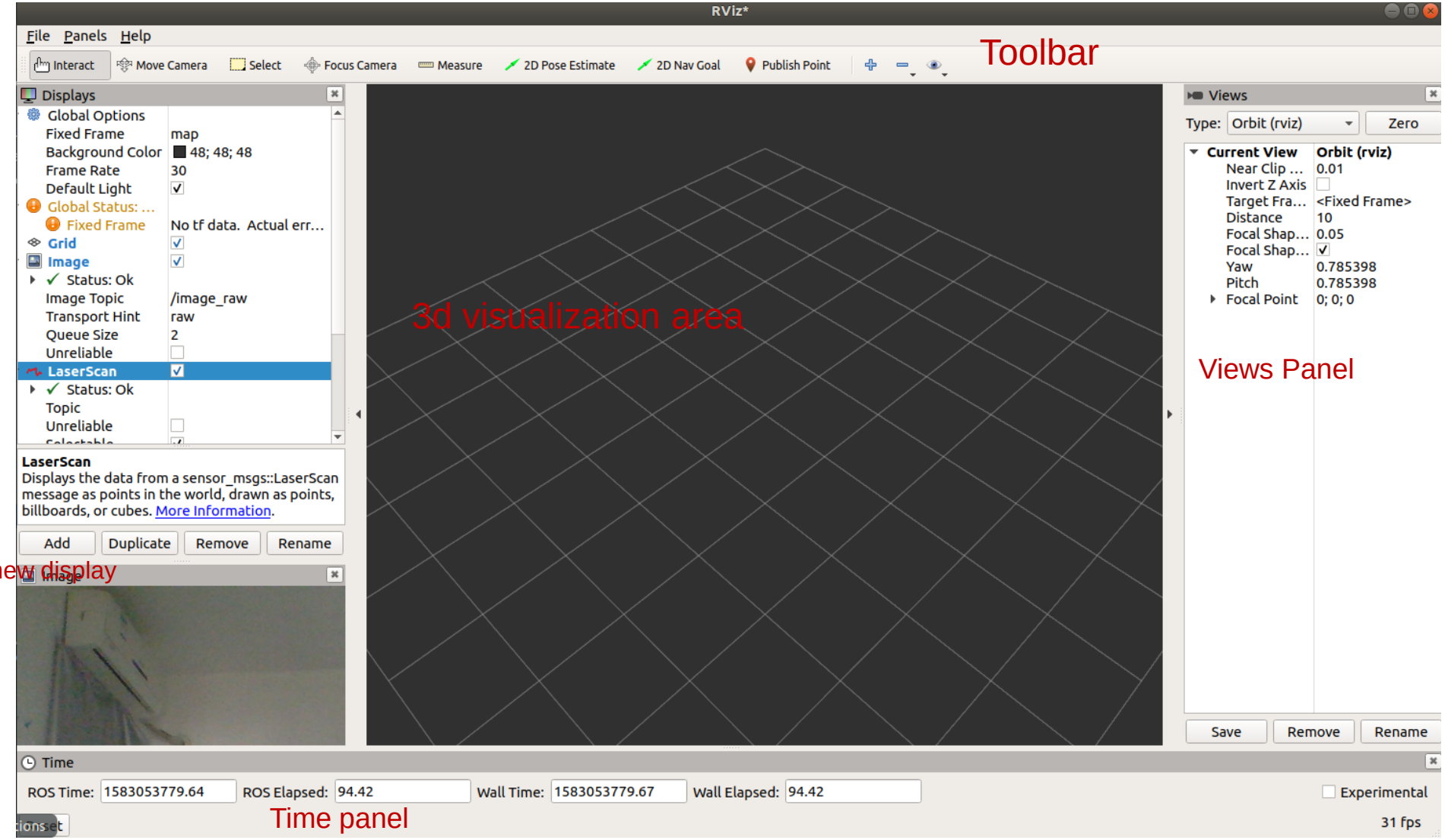
➢ Rviz

rviz is a 3d visualization environment for robots using ROS.

```
$rosrun rviz rviz
```

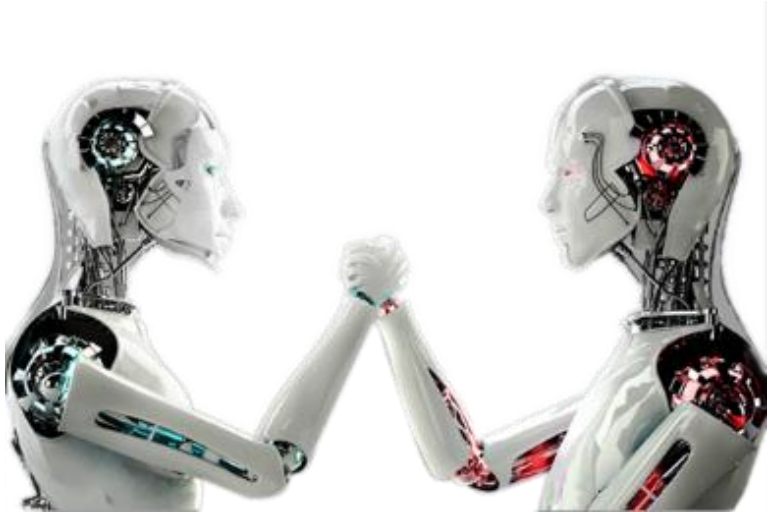
Display the list of property items

Adding a new display



PART FOUR

Lab Task



Lab Task

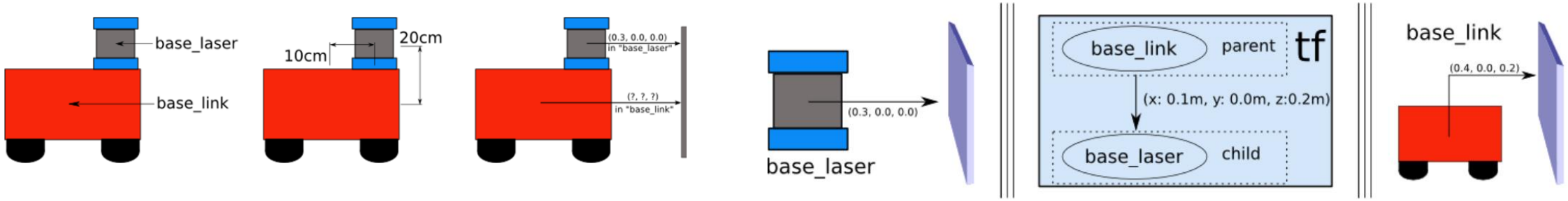
1.Complete the tf2 tutorials in *http://wiki.ros.org/tf2/Tutorials*

2.Complete the lab content of lab ppt ,and submit the zip file of learnning_tf package to BB .

Lab Task

3. **(Optional, bonus):** *As shown in the figure below, the coordinate relationship between the center point of the laser and the center point of the base of the robot is given,You are required to transforming the laser scan received from the "**base_laser**" frame to the "**base_link**" frame.*

*Please create broadcast and listening files(robot_tf_listener.cpp and rosbot_tf_broadcaster.cpp or robot_tf_listener.py and rosbot_tf_broadcaster.py) directly in the learning_tf package, and **submit*** **the zip file of learnning_tf package** ***to BB***.



Lab References

Futher References

- tf
  - http://wiki.ros.org/tf
- tf2
  - http://wiki.ros.org/tf2
- Rivz
  - http://wiki.ros.org/rviz
- roslaunch
  - http://wiki.ros.org/roslaunch
- Parameter Server
  - http://wiki.ros.org/Parameter%20Server

Thanks